

Strudel 小白指南

用代码写音乐：浏览器里的现场编程（Live Coding）乐器

一言以蔽之：写一行 JavaScript，浏览器立刻播放一段音乐；改一个字符，音乐当场变样。这就是 Strudel——Haskell 老牌作曲语言 TidalCycles 的 JavaScript 移植版。

热点来源：X 用户 @ozzyxs1a 发帖“其实你可以直接用 Claude 做音乐”（15万+浏览），评论区猜测用的正是 Strudel / TidalCycles 这类实时编码音乐工具。

🔥 热点溯源：Claude 真能做音乐吗？

能，而且链路比想象中简单：**Claude（或任何人）写出 Strudel 代码 → 浏览器 REPL 解析成“事件时间表” → Web Audio 引擎发声**。LLM 特别适合干这件事：音乐意图 → mini-notation 字符串，正好是“文本进、文本出”，而发声、节拍、音色全由 Strudel 负责。本指南所有代码都在本地 Node 真实跑通，并渲染出了真正可听的 .wav 文件。

Q1 “用 Claude 做音乐”到底是怎么回事？

热点解密

音乐从来没离开过“符号文本”：乐谱本身就是代码。Strudel 把乐谱简化成一行行 JavaScript 字符串，例如 `s("bd sd hh cp")` 就是一段四分音符鼓点。而写这些字符串不需要会乐器、不需要乐理功底，只需要会“描述节奏”——**这恰好是 LLM 最擅长的事：你说“来一段 90 BPM 的 lo-fi 鼓点”，它直接吐出一段 Strudel 代码，粘贴进 strudel.cc 就响**。推文里的视频大概率就是：Claude 生成代码 → strudel.cc 播放，全程不到一分钟。

Q2 Strudel 是什么？和 TidalCycles 什么关系？

身世

TidalCycles（简称 Tidal）：2014 年起，音乐家 Alex McLean 用 Haskell 写的“模式作曲语言”，在“algorave（算法锐舞）”现场演出圈用了十年，但安装要装 Haskell + SuperCollider 两套环境，劝退无数人。

Strudel：2022 年 Alex McLean 和 Felix Roos 用 JavaScript 重写的 Tidal——**打开 strudel.cc 网页就能用，零安装，自带声音引擎**。官方文档原话：Tidal “本身不发声”，要外挂 SuperCollider 的 SuperDirt；Strudel 则“完全在浏览器里自己发声”。上游仓库本在 GitHub（felixroos/strudel），现开发主力已迁往 Codeberg（uzu/strudel）。

Q3 不装任何软件，浏览器凭什么能出声？ 原理

靠浏览器内置的 **Web Audio API**（网页版的数字音频工作站）。Strudel 的分层架构如下——上半部分是“谱面”，下半部分是“音箱”：

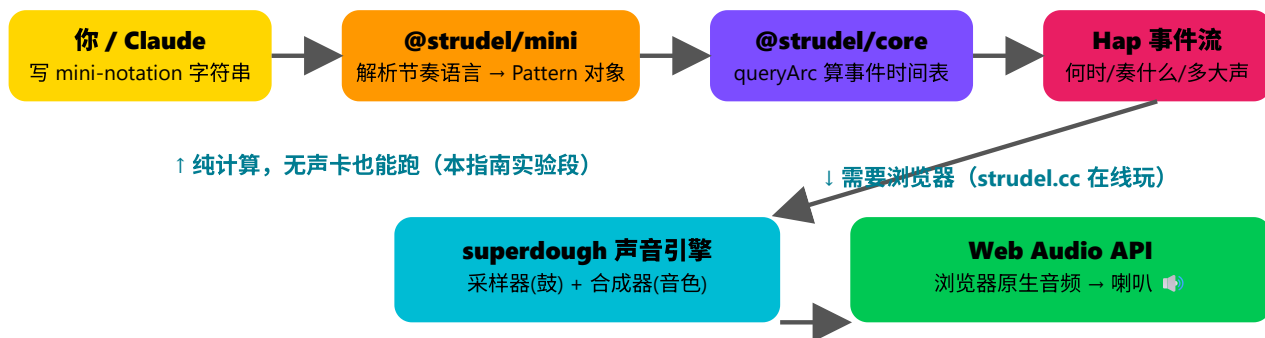


图1 Strudel 架构：谱面（可 headless 运行）与发声（浏览器 Web Audio）解耦

这个解耦是本次实验的关键：上半段（字符串 → 事件时间表）是纯数学，Node 里就能跑——所以我們能在无声卡的环境下验证“Claude 写的音乐逻辑对不对”，再渲染成 wav 亲耳验收。

Q4 mini-notation：一行字符串怎么变成一段节奏？ 核心语法

mini-notation 是 Strudel 的“谱面简谱”，符号极少但表达力极强（每个事件的真实输出见实验2）：

符号	含义	例子（实测输出）
空格	顺次排队，均分一拍	"bd sd bd sd" → 4 个事件各占 ¼ 循环
~	休止符（空一拍）	"bd ~ sd ~" → 只有 2 个事件
*n	循环内重复 n 次（提速）	"hh*4" → 半循环内挤 4 个踩镲
/n	放慢 n 倍	"bd/2" → 两循环才响一次
[]	分组：方括号整体占一格，内部再细分	"[bd sd] hh*2" → bd、sd 挤进前半拍
< >	交替：每循环换一个	"<bd sd> hh" → 第1循环 bd，第2循环 sd
(a,b)	欧几里得节奏：a 声尽量均匀散进 b 格	"bd(3,8)" → 0, 0.38, 0.75 处三声
c3 f#2	音符名 + 八度（C 调 12 平均律）	"c a f e" → 四个音高事件

bd=底鼓(kick)，**sd**=军鼓(snare)，**hh**=踩镲(hihat)，**cp**=拍手(clap)，**~**=rest——这套缩写沿用自 Tidal 的 Dirt 采样库。

Q5 "Live Coding 现场编程"与 algorave 是什么？

文化

Live coding (现场编码) 是一种表演艺术：音乐家/艺术家在观众面前**现场写代码**，大屏幕投影给观众看，代码改一行音乐当场变化——不编译、不重启、不打破节拍。**Algorave** 就是拿这种表演开派对的场景（2011 年始于伦敦），全球已有数百位活跃表演者。Strudel 的 REPL 就是为此设计的：代码热替换，鼓点从不中断。所以那条推文里的视频，本质是"一个人（或 Claude）在现场编程演奏"。

Q6 为什么说 Strudel 是"函数式"音乐？

核心思想

Pattern 不是音频，是"带时间坐标的函数"：给它一段时间区间，它返回这段时间内该发生的事件列表。于是变换函数可以对 pattern 做纯数学运算：

变换	语义	实测 ("bd sd hh cp", 2 循环)
<code>.fast(2)</code>	整体加速 2 倍	事件数 8 → 16, 坐标全部压缩一半
<code>.slow(2)</code>	整体放慢 2 倍	事件数 8 → 4 (铺满两倍时长)
<code>.rev()</code>	时间倒放	坐标 0,.25,.5,.75 → .75,.5,.25,0
<code>.add(3)</code>	音符整体移调 (加 3 个半音)	c a f e → midi 值 48/57/53/52 变 51/60/56/55
<code>.off(0.25, p)</code>	复制一份、错开 1/4 循环叠加	制作回声/卡农效果

这就是 Tidal 论文标题里的 "pattern as function of time"：因为 pattern 是函数，才能像数据一样被 map/reverse/zip——**把函数式编程的思想搬进了作曲。**

Q7 queryArc：字符串是怎么变成"何时发声"的？

事件模型

核心数据结构有两个：

Hap (事件) = `{ whole: 起止时间(分数), value: 内容 }`。时间用**分数 (Fraction)** 而不是小数——因为节奏是 1/2、1/4、1/8 不断二分/三分的，分数运算**永不丢精度**（实测输出 `a: 0 - 1/2, b: 1/2 - 3/4, c: 3/4 - 1`）。

queryArc(begin, end) = 问 pattern: "从第 begin 循环到第 end 循环，你有哪些事件？" pattern 返回 Hap 列表，声音引擎照单发货。**整个 Strudel 就是一个"时间 → 事件"的查询系统**；我们实验里渲染 wav，正是消费同一个 queryArc 输出。

Q8 欧几里得节奏 bd(3,8) 是什么原理？ 彩蛋算法

把 3 声"尽量均匀"地撒进 8 格——用欧几里得 GCD 算法分配间隔。实测 `bd(3,8)` 落在 0、0.38、0.75：正是古巴 clave 节奏的骨架。有趣的事实：这类"最大均匀分布"节奏遍布非洲、中东、electronic 音乐——**一条数学公式统一了世界各地的传统节奏**，这是 Tidal 最出名的卖点之一。

Q9 怎么叠出一整首歌？ stack 编曲

`stack`(鼓，镲，贝斯，琶音) 把多个 pattern 纵向叠加同时播放——就是多轨录音机的代码版。我们主实验的"谱子"总共 5 行：

```
// 实验4的歌曲：5 个 pattern 叠成 4 循环 lo-fi 小段
stack(
  mini('bd ~ bd ~ bd ~ bd bd'),           // 底鼓
  mini('~ sd ~ sd ~ sd ~ sd'),           // 军鼓踩 2/4 拍
  mini('hh*8'),                           // 踩镲八连音
  mini('<c3 e3 g3> <a2 c3 e3> <f2 a2 c3> <g2 b2 d3>'), // 每循环换和弦
  mini('<c2 g1>*2 <a1 f1>*2'),             // 贝斯跟根音
)
```

渲染结果 100 个事件，钢琴roll 视图（真实数据，非示意图）见实验章节图3。

Q10 AI (Claude) 在这条链路里扮演什么角色？ AI × 音乐

注意：**Strudel 本身不用 AI**——它是个确定性的音乐引擎。AI 的位置在"写谱"那一端：

环节	谁来干	为什么合适
音乐意图 → mini-notation 代码	LLM	文本进文本出；训练语料含大量 Tidal/Strudel 教程
代码 → 精确事件时间表	Strudel 引擎	确定性计算，毫厘不差
事件 → 声音	Web Audio / 我们的自制合成器	DSP 信号处理
听感验收 → 改代码	人（或 AI 循环自纠）	审美闭环；AI 可配合"生成→查询事件→自查节奏密度"循环

所以"用 Claude 做音乐"的准确说法是：**Claude 当作曲，Strudel 当乐队，浏览器当音箱。**

Q11 这个项目给我什么能力? 能力清单

- **能1 零安装作曲**: 打开 `strudel.cc`, 一行代码出声, 改一个字符变奏——最适合编程入门者的"能听见反馈"的 playground。
- **能2 算法作曲**: 用欧几里得节奏、随机信号(perlin/sine)、pattern 变换批量生成人手写不出的复杂节奏。
- **能3 现场表演/冥想伴奏**: live coding 改 pattern 不中断节拍, 可以直接做短视频 BGM、专注背景音乐。
- **能4 音乐逻辑 headless 验证**: 事件时间表在 Node 里可查询可断言——可以写单元测试测"歌对不对", 甚至让 AI 自查。
- **能5 离线渲染音频文件**: 像实验4那样把 pattern 渲染成 wav/mp3, 脱离浏览器产出成品。
- **能6 理解 Web Audio 与 DSP 入门**: 采样、合成、包络、混音——做音乐顺便入门数字信号处理。

Q12 坑与现状 (2026-10 实测) 避坑

坑	说明与解法
npm 发布包失修	<code>@strudel/core@1.2.x</code> 的 dist 依赖 <code>@kabelsalat/web</code> 的 <code>SalatRepl</code> 导出, 而 0.3.x/0.4.x 都没有 → Node 里一 import 就 <code>SyntaxError</code> 。 解法 : 不用 npm 包, 直接用仓库源码 (<code>packages/core + packages/mini</code> , 仅依赖 <code>fraction.js</code>)。
开发已迁 Codeberg	GitHub 仓库仍在但开发主力在 <code>codeberg.org/uzu/strudel</code> , issue/PR 去那边提。
Node 端警告	源码引入时打印 "cannot use window: not in browser?"——无害, pattern 查询不受影响。
时间用分数	事件坐标是 Fraction 对象, 打印/画图前要 <code>Number(h.whole.begin)</code> 转换, 别直接当 float。
音高≠频率	"c3" 到频率要按 A4=440Hz 换算 ($c3 \approx 130.8\text{Hz}$), 自制渲染器别把 c3 当 3Hz。

Q13 上手路径怎么走? 学习路线

路线一 (5 分钟爽): 打开 `strudel.cc` → 点左上 "go to REPL" → 输入 `s("bd sd hh cp")` → Ctrl+Enter 播放 → 随手改字符串听变化。

路线二 (系统学): 官方 Workshop (`strudel.cc/workshop`) 四部曲: First Sounds → First Notes → First Effects → Pattern Effects, 每页都内嵌可演奏的 MiniREPL。

路线三 (程序员向): 克隆仓库, 读 `packages/core/pattern.mjs` (约 1300 行, 整个模式引擎) + 本指南实验目录的 4 个脚本——从"用引擎"到"懂引擎"。



动手实验：从一行代码到一首 6 秒的歌（全部真实运行）

环境：Windows 11 + Node v24.21.0，无任何音频设备参与。实验材料在 `F:\reminder\strudel\exercise\`。

备 步骤 0：环境准备（一次性）

为什么/操作

```
# npm 发布包有坏依赖（见 Q12），我们直接把仓库源码 vendor 进来
git clone --depth 1 https://github.com/felixroos/strudel.git repo
mkdir -p node_modules/@strudel
cp -r repo/packages/core node_modules/@strudel/core # 模式引擎
cp -r repo/packages/mini node_modules/@strudel/mini # 节奏语言解析器
npm i fraction.js # 唯一外部依赖：分数运算
```

✅ **预期结果：** `node -e "import('@strudel/mini').then(m=>console.log('ok'))"` 打印 
`@strudel/core loaded` （附带一条无害的 window 警告）

实1 实验 1（冒烟）：引擎真的能在 Node 里转吗？

命令与目的

```
node smoke.mjs # 最小验证：pattern 引擎 + mini 解析器
```

smoke.mjs 内容： `sequence('a', ['b', 'c']).queryArc(0,1)` 和 `mini('c a f e').queryArc(0,1)`

✅ **真实输出：** `a: 0 - 1/2, b: 1/2 - 3/4, c: 3/4 - 1`（嵌套数组自动均分！）+ `"c" "a" "f" "e"` 四个音符事件。引擎在无声卡环境完全可运行。

实2 实验 2：mini-notation 八连测——每种符号都眼见为实

命令

```
node 01_mini_notation.mjs
```

✓ 真实输出 (节选) :

```
mini("bd*2 sd hh*4") → bd@0.00~0.17 bd@0.17~0.33 sd@0.33~0.67 hh@0.67~0.75 ...hh*4  
(* 重复压缩时长)  
mini("<bd sd> hh") → cycle0: bd@0.00~0.50 / cycle1: sd@0.00~0.50 (<> 逐循环交替)  
mini("bd(3,8)") → bd@0.00~0.13 bd@0.38~0.50 bd@0.75~0.88 (欧几里得 3 声均匀撒 8 格)
```

实3 实验 3：函数式变换——同一个 pattern 的四种人生

命令

```
node 02_transforms.mjs
```

✓ 真实输出: fast(2) 事件 8→16 (坐标减半) ; slow(2) 8→4; rev() 坐标倒序; add(3) 音符变 midi 数字 51/60/56/55; stack(鼓, 镲, 贝斯) 单循环 13 事件叠加。验证了"pattern 是时间的函数, 变换是函数的组合"。

实4 实验 4（主实验）：把代码渲染成真正可听的 .wav

怎么操作 → 目的 → 检测

```
node 03_render_wav.mjs
```

目的：脱离浏览器，证明 Strudel pattern 是完整的“音乐源代码”。脚本做的事：queryArc 拿到 100 个事件 → 自制合成器（底鼓=130→45Hz 滑频正弦、军鼓=噪声+180Hz、踩镲=白噪声、音符=谐波+包络）→ 写 44100Hz/16bit/单声道 WAV。

✓ **真实输出：**strudel_jam.wav 生成：516.8 KB，6.0 秒（底鼓×20 军鼓×16 踩镲×32 和弦音×9 贝斯音×23）

✓ **Python wave 模块独立验证：**mono/44100Hz/16bit，峰值 29163（≈0.89 满幅），首秒 96.6% 样本非静音——**双击播放，是一段真实可听的 6 秒 lo-fi 鼓点小段。**

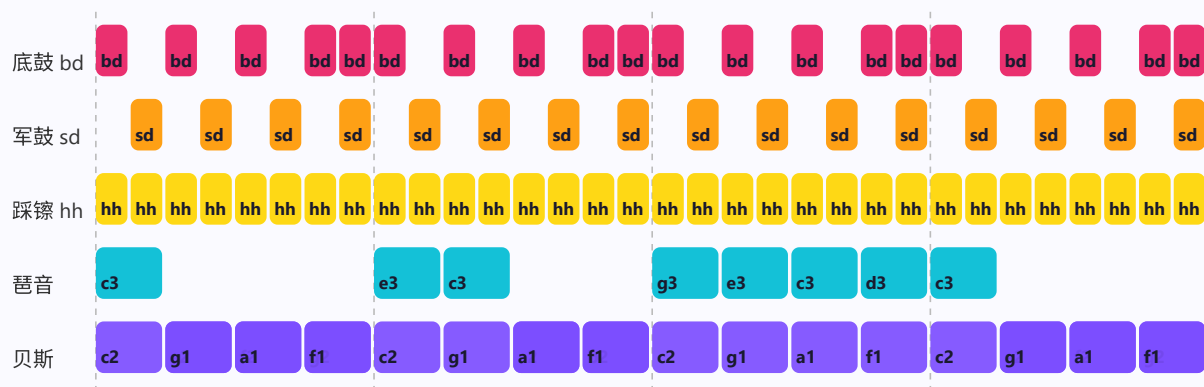


图3 实验4歌曲的钢琴roll (04_pianoroll_svg.mjs 由 100 个真实事件坐标生成，非示意图)

实5 实验 5：浏览器侧验证 + 真实出声 (strudel.cc)

怎么操作

Edge 无头浏览器访问官方文档页截图成功 (exercise\strudel_repl.png)，确认 strudel.cc 在线可用；**建议你自己打开 strudel.cc 粘贴实验4的 stack() 五行代码，戴耳机按 Ctrl+Enter——同样的谱子，浏览器用真采样/合成器演奏，比我们的极简合成器动听得多。**

✓ **预期结果：**REPL 顶部波形动画滚动，听到鼓+贝斯+琶音齐奏；把 `<c3 e3 g3>` 改成 `<c3 f3 a3>`，和弦进行当场改变而节拍不中断——这就是 live coding。

来源与延伸：官网 strudel.cc (REPL/Workshop/文档) · 上游仓库 github.com/felixroos/strudel (开发已迁 codeberg.org/uzu/strudel, AGPL-3.0) · TidalCycles 原版 tidalcycles.org · 热点推文

